SKILL WEEK:- 5

Task 1:-

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

Source Code :-

```
  GNU nano 8.7.1                single_quotes.c *
#include <stdio.h>
#include <string.h>

#define MAX 1000

void parse_single_quotes(char *input)
{
    char result[MAX];
    int i = 0, j = 0;
    int in_single = 0;

    while (input[i] != '\0')
    {
        if (input[i] == '\'')
        {
            in_single = !in_single;
            i++;
            continue;
        }

        result[j++] = input[i++];
    }

    result[j] = '\0';

    if (in_single)
    {
        printf("Error: Unmatched single quote.\n");
    }
    else
    {
        printf("Parsed Result: %s\n", result);
        printf("Single quote parsing successful.\n");
    }
}

int main()
{
    char input[MAX];

    printf("Single Quote Parser\n");
    printf("-------------------\n");

    printf("Enter a string: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    parse_single_quotes(input);

    return 0;
}
```

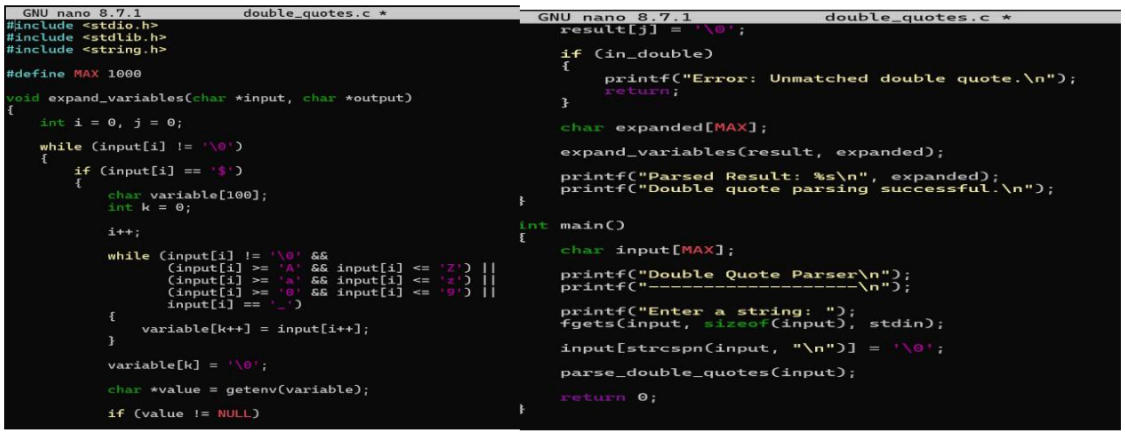
Output:-

```
sathwika@DESKTOP-AA8K5MG:~$ nano single_quotes.c
sathwika@DESKTOP-AA8K5MG:~$ gcc single_quotes.c -o single_quotes
sathwika@DESKTOP-AA8K5MG:~$ ./single_quotes
Single Quote Parser
-------------------
Enter a string: Hello World
Parsed Result: Hello World
Single quote parsing successful.
sathwika@DESKTOP-AA8K5MG:~$
```

Task 2:-

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

Source Code:-



```
 GNU nano 8.7.1                double_quotes.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 1000

void expand_variables(char *input, char *output)
{
    int i = 0, j = 0;

    while (input[i] != '\0')
    {
        if (input[i] == '$')
        {
            char variable[100];
            int k = 0;

            i++;

            while (input[i] != '\0' &&
                   (input[i] >= 'A' && input[i] <= 'Z') ||
                   (input[i] >= 'a' && input[i] <= 'z') ||
                   (input[i] >= '0' && input[i] <= '9') ||
                   input[i] == '_')
            {
                variable[k++] = input[i++];
            }

            variable[k] = '\0';

            char *value = getenv(variable);

            if (value != NULL)
```

```
GNU nano 8.7.1                double_quotes.c *
            result[j] = '\0';

        if (in_double)
        {
            printf("Error: Unmatched double quote.\n");
            return;
        }

        char expanded[MAX];

        expand_variables(result, expanded);

        printf("Parsed Result: %s\n", expanded);
        printf("Double quote parsing successful.\n");
}

int main()
{
    char input[MAX];

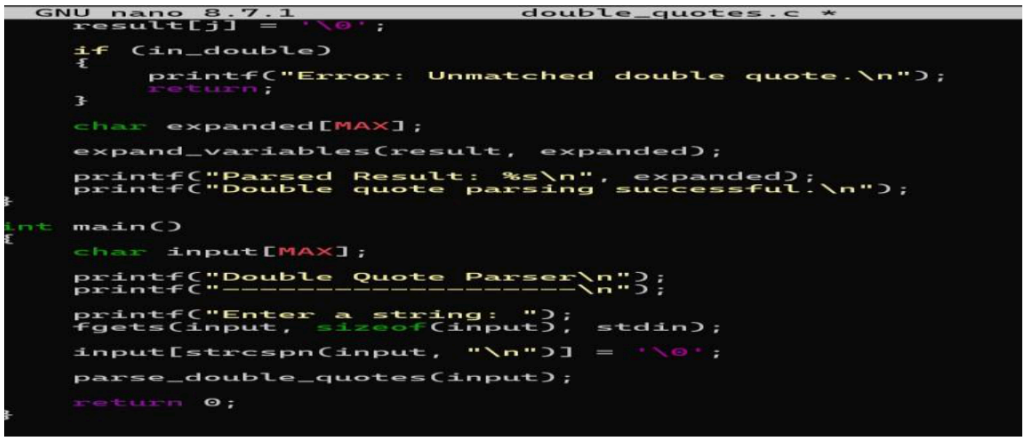
    printf("Double Quote Parser\n");
    printf("--------------------\n");

    printf("Enter a string: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    parse_double_quotes(input);

    return 0;
}
```



```
 GNU nano 8.7.1                double_quotes.c *
    result[j] = '\0';

    if (in_double)
    {
        printf("Error: Unmatched double quote.\n");
        return;
    }

    char expanded[MAX];

    expand_variables(result, expanded);

    printf("Parsed Result: %s\n", expanded);
    printf("Double quote parsing successful.\n");
}

int main()
{
    char input[MAX];

    printf("Double Quote Parser\n");
    printf("--------------------\n");

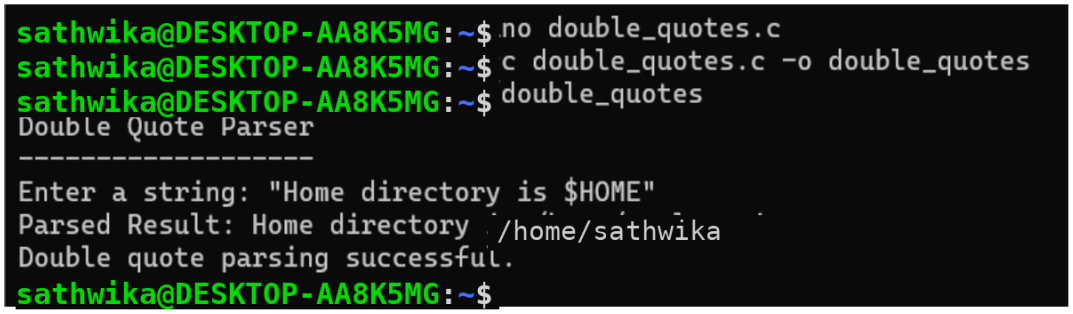
    printf("Enter a string: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    parse_double_quotes(input);

    return 0;
}
```

Output:-



```
sathwika@DESKTOP-AA8K5MG:~$ no double_quotes.c
sathwika@DESKTOP-AA8K5MG:~$ c double_quotes.c -o double_quotes
sathwika@DESKTOP-AA8K5MG:~$ double_quotes
Double Quote Parser
--------------------
Enter a string: "Home directory is $HOME"
Parsed Result: Home directory is /home/sathwika
Double quote parsing successful.
sathwika@DESKTOP-AA8K5MG:~$
```